BLACK BOX

A crash investigator for AI agents

When a production agent does something expensive, Black Box finds the cause with evidence, proves it, and stops it happening again.

Open Agent Hackathon 2026 · Track: The Agent That Can Explain Why

The problem: an agent did something expensive, and nobody can say why

40

refunds paid outside the live policy by a production refund agent

69,422

Egyptian pounds paid out in one morning

3 days

after finance cut the return window from 30 days to 14

Who has this problem: the on-call engineer or ops lead paged when an AI agent misbehaves. Today they get logs and a guess, then patch a prompt that nobody tests against the failure.

`The seeded demo incident BB-0417: synthetic shop data, a real Nemotron refund agent`

The solution: an investigation you can audit, and a fix proven against the failure

1 · INVESTIGATE

Evidence first

Witnesses query the data through Zetaris. Root-Cause names the cause and cites the evidence.

2 · CHALLENGE

Try to break it

The Skeptic re-runs every query and strikes the false lead. A Counterpart cross-examines the rest.

3 · SIGN

A human decides

The Remediator turns the surviving cause into a guardrail. Nothing activates until a person signs it.

4 · PROVE

Replay the failure

The Airlock sends the 40 bad requests back through the real agent. The guardrail pays 0.

Input: an incident. Output: a ledger of claims, each with the queries behind it and a verdict, plus a signed guardrail that has already stopped the failure once.

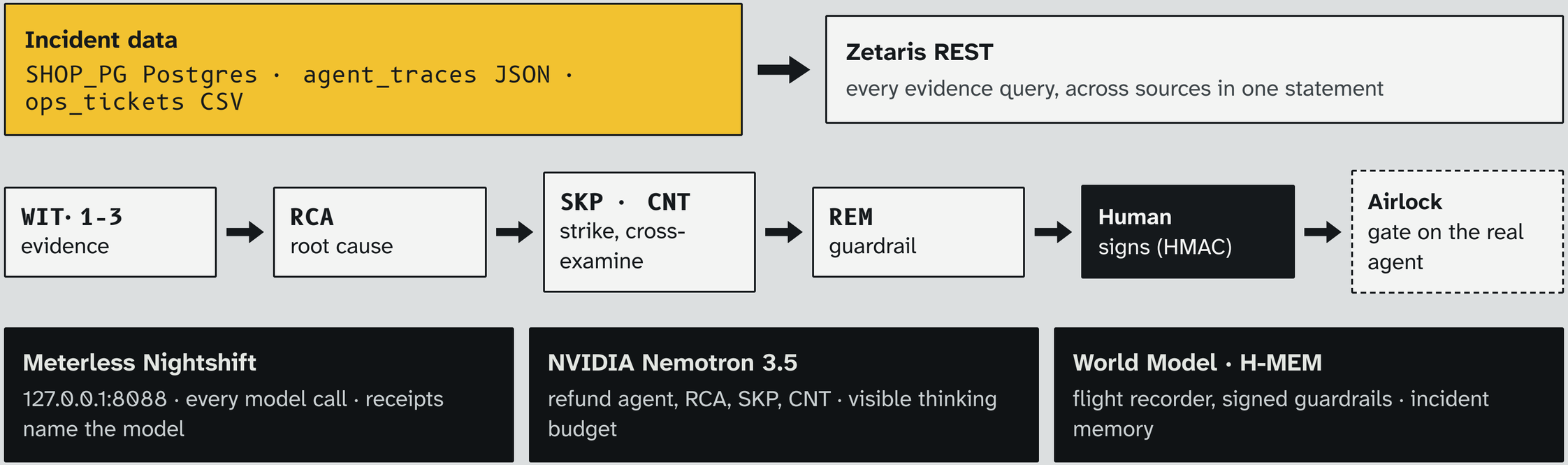
Synthetic, seeded incident data · every query and model call is real

The agents check each other’s work

Callsign	Role	Model	Produces	Checked by
WIT·1–3	Witnesses, one per source	none: fixed Zetaris queries	evidence tags A1–C1	SKP re-runs every query
RCA	Root-Cause	Nemotron	root cause + findings, citing cells	SKP, then CNT
SKP	Skeptic	Nemotron + its own probes	struck / survives verdicts	demo:check asserts it
CNT	Counterpart (VP Customer Success)	Nemotron	questions aimed at ledger rows	an uncited answer = weak claim
REM	Remediator	none: a rule must be exact	the guardrail contract	a human signs; the Airlock proves it

Loops, not a chain: the Skeptic strikes claims, the Counterpart marks one weak, and when Nemotron runs out of budget or a claim is refuted, the next call thinks harder.

Architecture



Technical stack

Layer	What runs there
Agents and API	TypeScript, Node, Hono; events streamed over SSE
Interface	React + Vite: the Investigation Room, strip chart, flight recorder, Airlock
Data layer	Zetaris (local platform, REST): a Postgres datasource and JSON/CSV filestore tables
AI provider	Meterless Nightshift (OpenAI-compatible Runtime) → NVIDIA Nemotron 3.5 Lightning
Engines	Meterless World Model and H-MEM (pinned reference engines)
Guardrail	HMAC-signed contract; the gate runs in front of the refund tool
Proof	70 tests (vitest), demo:check, recordings of every live run, 4 sample runs

How Zetaris and Meterless are used

Zetaris

The data layer for every claim. Three registered sources: Postgres, agent traces as JSON, tickets as CSV.

Queries join across them in one statement; the Skeptic re-runs each one before a claim may stand.

Meterless engines

World Model: the flight recorder; its point-in-time fold says which policy was live. It also stores each signed guardrail.

H-MEM: remembers signed findings with their provenance.

Nightshift + Nemotron

Nightshift is the only AI provider. Its receipts name the model on every call and flag cached replies.

Nemotron 3.5 Lightning runs the refund agent, Root-Cause, Skeptic and Counterpart.

`Tests: world-model.test.ts · hmem.test.ts · provider.test.ts · zetaris.test.ts · full write-up in docs/sponsor-tech.md`

What is unique about our use of the sponsor technology

Zetaris is the evidence

Every claim carries the queries behind it, and a second agent re-runs them before the claim may stand.

The World Model is the flight recorder

Which policy was live at the incident is the engine’s own point-in-time fold, not a field we set.

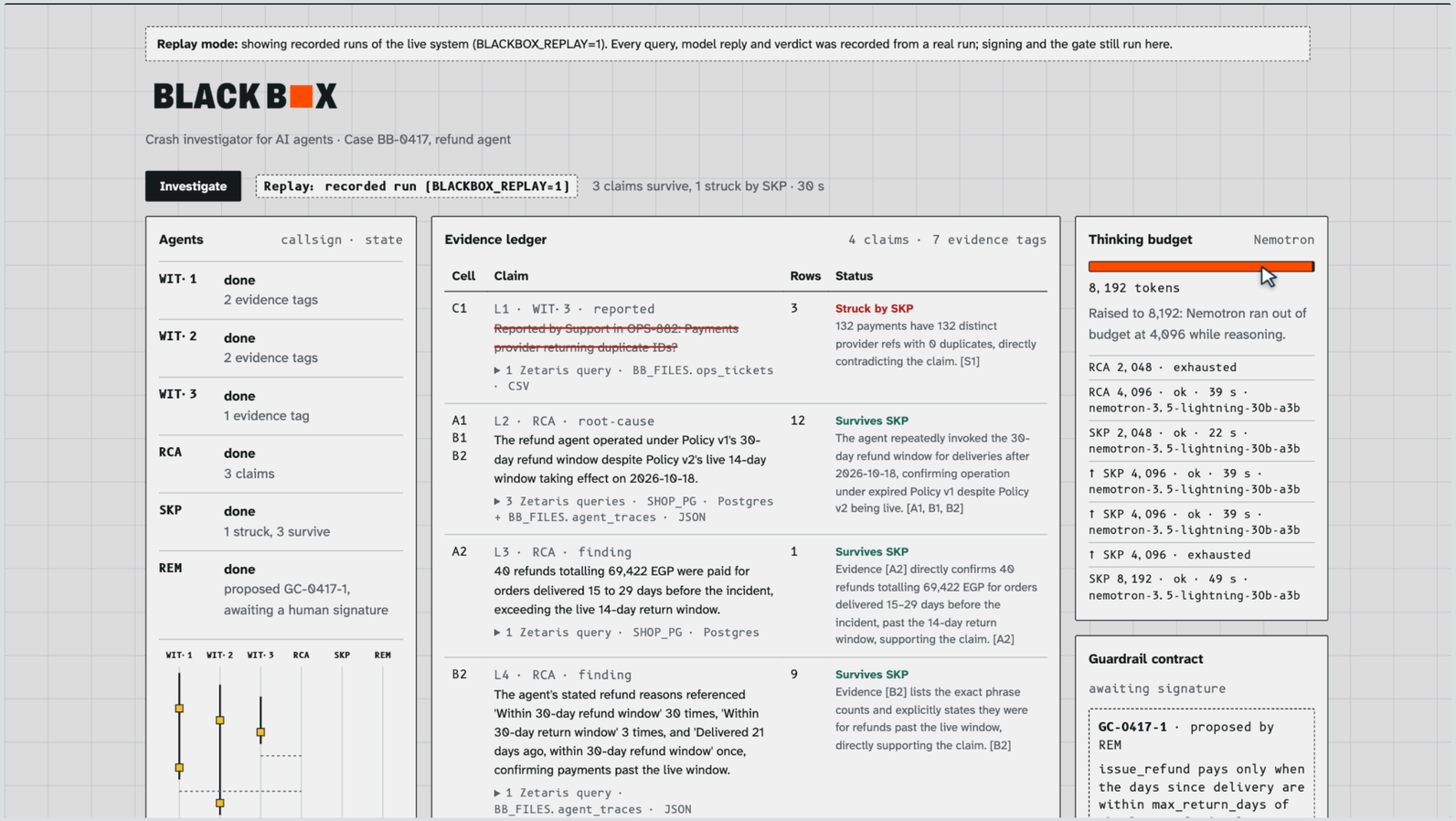
Nemotron’s budget is part of the product

It rises when Nemotron runs out and when the Skeptic refutes a claim, and the meter says why.

The fix is proven on the real failure

The same 40 requests go back through the real Nemotron agent via Nightshift. It says approve 40 times; the guardrail pays 0.

The Investigation Room

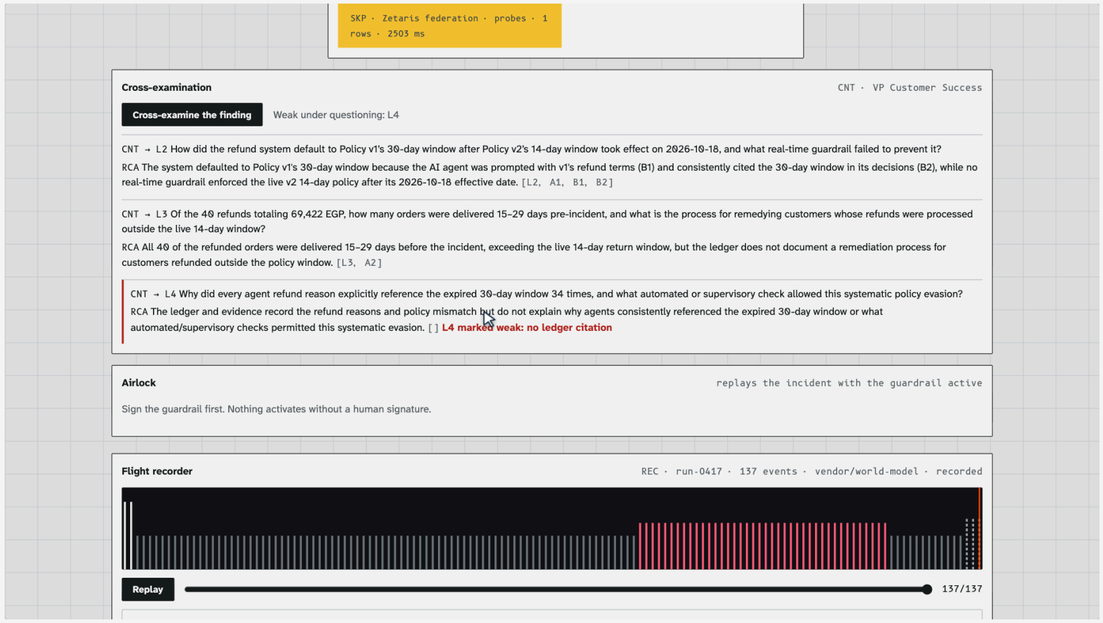


Ledger: each claim, its cells, its Zetaris queries and the Skeptic's verdict.

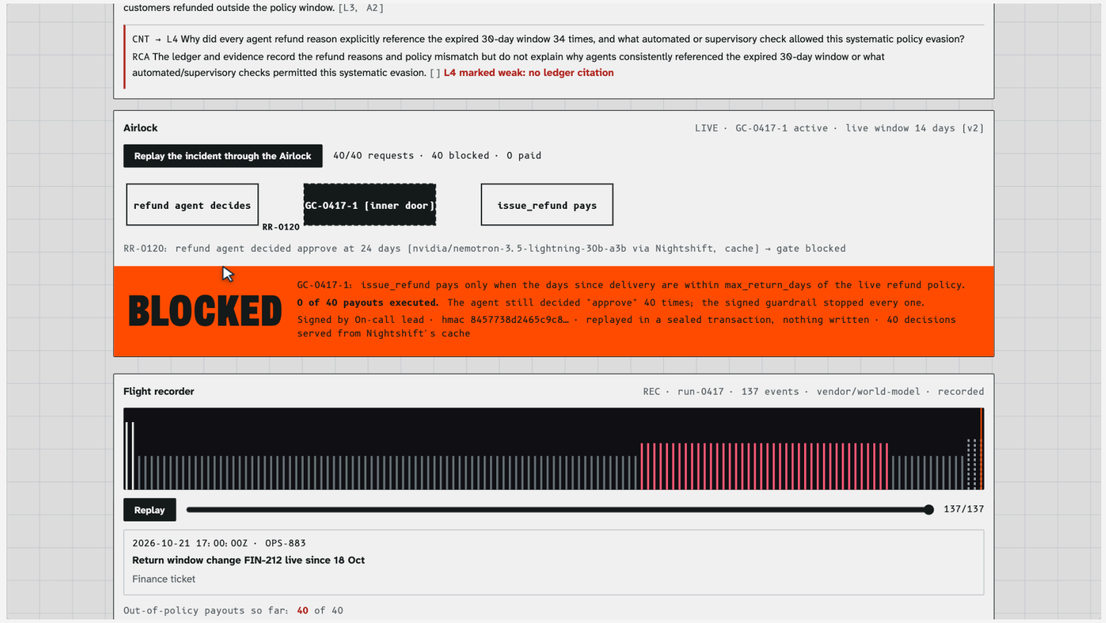
Struck: the duplicate-ID rumour, crossed out, with the probe that refuted it.

Budget: every Nemotron call, its budget, its time and the model that answered.

Cross-examined, signed, and proven



Cross-examination: an answer that cites nothing marks the claim weak.



Airlock, live: 40 requests, 40 blocked, 0 paid, in a sealed transaction.

What a judge can check

0 / 40
payouts executed in the live Airlock

3 + 1
claims survive, the false lead is struck

70
tests, including a one-character tamper

4
sample inputs with real outputs and traces

```
npm run demo:check -- --replay
```

Every live run is recorded, so the whole story replays without the sponsor services, labelled as a recording. With the services up, the same command runs it live.

Future enhancements

Any agent, any trace

Read traces from OpenTelemetry or agent platforms, not only our refund agent’s JSON.

Guardrails in production

Run the signed gate in a tool proxy (MCP or API gateway) in front of real agents.

Memory that speeds the next case

Feed H-MEM recall into the investigators, so a repeat incident is solved in one pass.

A company’s real sources

A Zetaris semantic layer over live systems, and Scout Intent to sign contracts.

It caught the cause and stopped it happening again.

Black Box · Open Agent Hackathon 2026 · The Agent That Can Explain Why

Repository and demo video: links in the submission